NexLink

AI-Assisted Private Networking, Network Diagnostics and Remote Administration Platform

Project Report

What is built, what is verified, and what remains

Item	Detail
Prepared for	Maitrey Bharambe
Date	24 October 2026
Repository	github.com/Maitrey-Bharambe/nexlinkk
Versions	Web app 0.3.0 · networking agent 0.2.0
Stage	One deployable app (one process, one port); real WireGuard tunnels verified on one PC over the direct path and the encrypted relay
Basis of this report	Test runs and live checks performed on 16–24 October 2026. Nothing is reported as done unless it was tested; every figure names the run it comes from.

Contents

1. Executive summary 3
Key numbers 3
2. Goals and scope 4
Technology stack 4
3. Architecture 5
Components 5
Four layers of “is it connected?” 6
Enrollment flow: join with a code 6
Paths: direct first, relay when needed 6
Peer link states 7
Diagnostic loop 7
4. What is in the project 8
4.1 Public pages 8
4.2 Console 9
4.3 Server (inside the web app) 10
4.4 Networking agent (Go) 10
4.5 Diagnostic service (Python, optional) 11
5. Status matrix 12
6. Verification results 13
6.1 Automated tests 13
6.2 Real WireGuard tunnel with real names 13
6.3 Browser checks 13
6.4 Bugs found and fixed during verification 14
7. Security model 15
8. What is not in the project yet 16
Known limitations and environment notes 16
9. How to run 17
Roadmap 17
Appendix A — Repository layout 18
Appendix B — Documentation index 18

1. Executive summary

NexLink is a private-networking platform that only reports what it can prove. Operators create a private network, add computers with a short code, and see — per device and per peer pair — whether WireGuard has actually completed a handshake. Diagnostics and AI analysis explain problems from recorded evidence; any action they propose passes a policy engine and needs a human to approve it.

Since the 19 October report the architecture was rebuilt so the whole platform is **one Next.js app on one port**: console pages, REST API, Socket.IO live updates and an **encrypted relay** for devices that cannot reach each other directly. It ships as one container plus MongoDB. Enrollment now uses a **device code** (one line on the computer, a short code approved in the console), AI briefings and incident analysis are **built in** (Groq), and the interface was redesigned with a public landing page, a new sign-in page and a consistent pixel-display design system.

On 22 October two real agents were enrolled from the console under real names (**maitrey** and **maitrey-peer**, network **maitrey-net**) and brought up with the production `run --userspace` command. Both the relay path and the direct UDP path completed WireGuard handshakes and carried traffic; the console reported the network **OPERATIONAL — 1/1 peer pairs verified by fresh handshake**. The same tunnel was re-checked on 24 October.

Key numbers

Measure	Result	Notes
Automated tests passing	128 / 128	Web app 85 (24 Oct), Go agent 28 (24 Oct), diagnostic service 15 (24 Oct)
Production build	Passed	Next.js 15, all 19 routes, one server process (22 Oct)
Device-code enrollment, real agent	Verified	maitrey and maitrey-peer joined maitrey-net; codes approved on “add a device” (22 Oct)
WireGuard tunnel — direct UDP	Verified	Handshake within 1 s; RTT through the tunnel 0.35 ms (22 Oct) and 0.42–0.48 ms (24 Oct)
WireGuard tunnel — encrypted relay	Verified	Handshake within 1 s; probes 0.99–3.49 ms through the relay (22 Oct)
Console verdict for maitrey-net	Operational	“1/1 peer pairs verified by fresh handshake”; both devices healthy
AI (built-in Groq provider)	Verified	Per-network briefings and incident analysis, each finding citing recorded evidence; disallowed actions rejected by the policy engine
Approved connectivity test	Successful	run_connectivity_test approved, executed by the agent, 3/3 measurements valid

Where the project stands

Done and tested: accounts and roles, networks, device-code enrollment and invite links, signed agent protocol, honest health model, topology, network observatory, incidents and the diagnostic loop, policy-gated actions with approval and verification, audit log, guided setup, encrypted relay, built-in AI briefings and diagnosis, landing and sign-in pages, redesigned console.

Ready, manual run pending: a tunnel between two physical computers, including across different networks through a deployed portal; remote sessions end to end; the kernel WireGuardNT interface via the WireGuard app.

Known gap: when two devices start on the relay at the same moment, a working relay link is not upgraded to direct later (details in section 8).

Not started: MFA, single sign-on, password reset, OS-keychain key storage, CI pipeline, application-level checks, multi-instance scaling.

2. Goals and scope

The brief asked for a serious Computer Networks and AI-systems project rather than a dashboard. The design follows these rules, and the tests check them:

- **Control plane and data plane are separate.** The server coordinates keys, addresses and policy; VPN packets travel between devices over WireGuard — directly when possible, otherwise through a relay that only ever sees ciphertext.
- **No fabricated state.** Missing data is shown as unknown or not measured, never as zero or healthy. The public landing page shows no invented customer numbers; its two measured figures name the run they come from.
- **Identity is cryptographic.** Devices prove possession of their own Ed25519 key; private keys never leave the device.
- **AI is advisory and gated.** Analyses must cite collected evidence; actions come from an allowlist, pass a deterministic policy engine, and need human approval. Briefings are explain-only.
- **Host changes are opt-in.** The agent is read-only unless privileged mode is enabled locally on that machine.
- **One app to deploy.** One process on one port and one database, so a single container can run the whole platform.

Technology stack

Layer	Technology	Role
Web app	Next.js 15 (App Router, JavaScript), React 19, Tailwind v4, React Flow, lucide-react; custom `server.js` hosting Express 4, Socket.IO and the WebSocket relay	Landing, sign-in and console pages; REST API; live updates; relay — all on port 3000
Design system	Press Start 2P (display), Space Grotesk (interface), JetBrains Mono (technical values); colour and status tokens	One type system across every page; status colours mean state, never decoration
Database	MongoDB (Atlas, Docker or an embedded dev instance)	State, with unique / partial / TTL indexes as integrity constraints
Networking agent	Go, wireguard-go + gVisor netstack, wgctrl, WebSocket relay client	Identity, signed transport, userspace tunnel, path selection, probes, approved commands, sessions, Windows service
VPN	WireGuard	Encrypted data plane (no custom cryptography)

Layer	Technology	Role
AI	Built-in Groq provider (openai/gpt-oss-120b, JSON mode); optional Python FastAPI service	Network briefings, incident diagnosis, device analysis — validated against schema and evidence

3. Architecture



Figure 1 — Components. The thick lime link is the preferred path for VPN traffic; the amber path is the relay fallback.

Components

Component	Location	Responsibility
Pages	web/app	Public landing (/welcome), sign-in (/login), join pages and the console; same-origin calls to /api/v1
User API	web/server/modules/*	Sessions (HttpOnly cookie), CSRF, RBAC, scoped access, strict validation
Agent API	web/server/modules/agent	Separate auth path: Ed25519-signed requests, timestamp window, single-use nonces
Enrollment	modules/enrollment	Device codes (proof of possession, 15-minute codes, operator approval) and invite links
Relay	modules/relay	Forwards WireGuard ciphertext between devices of one network; rate-limited; closed on revocation
Incident detector	modules/incidents	Evaluates heartbeats, interface state, handshakes, loss and latency every 30 s
Policy engine	modules/policies	Allowlist, parameter schemas, device state, capability, rate limits, approver permission
AI	modules/diagnostics, modules/insights	Built-in Groq provider; briefings per network (scheduled and on request); grounding validator
Networking agent	networking-agent/	join, run --userspace, path selection (direct → relay), probes, approved commands, sessions, service

Four layers of “is it connected?”

None of these is inferred from the layer above it:

Layer	Meaning	Evidence used
1 Registered	Identity key enrolled	devices.enrollmentState = enrolled
2 Configured	Desired peers computed	networks.configVersion and GET /agent/config
3 Tunnel up	WireGuard completed a handshake	Last handshake ≤ 180 s at report time
4 Reachable	Packets cross the tunnel	TCP probe to the peer’s tunnel address

Application-level checks (does a service on the peer answer?) are not measured yet.

Enrollment flow: join with a code

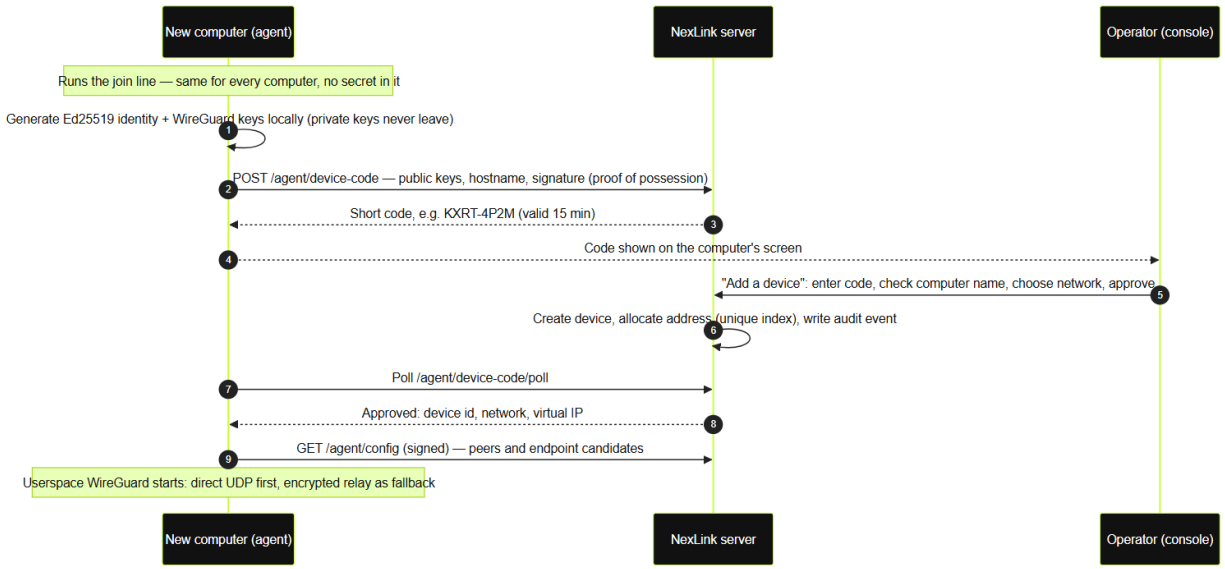


Figure 2 — Device-code enrollment: keys created on the computer, proof of possession, operator approval.

The join line is the same for every computer and contains no secret: `irm 'https://<portal>/join.ps1' | iex` on Windows or `curl -fsSL 'https://<portal>/join.sh' | sh` on macOS and Linux. Invite links remain available and fall back to a code when they expire.

Paths: direct first, relay when needed

Each peer is tried over direct UDP first. If no handshake arrives within 20 seconds the agent switches that peer to `relay:<device id>`: WireGuard packets travel inside the agent’s signed WebSocket to the server, which forwards them to the peer. The relay never holds a WireGuard key and only connects devices of the same network. The topology and device pages show **direct** or **relay** per link.

Peer link states



Figure 3 — States the topology draws; “configured” is never shown as a working tunnel.

Diagnostic loop

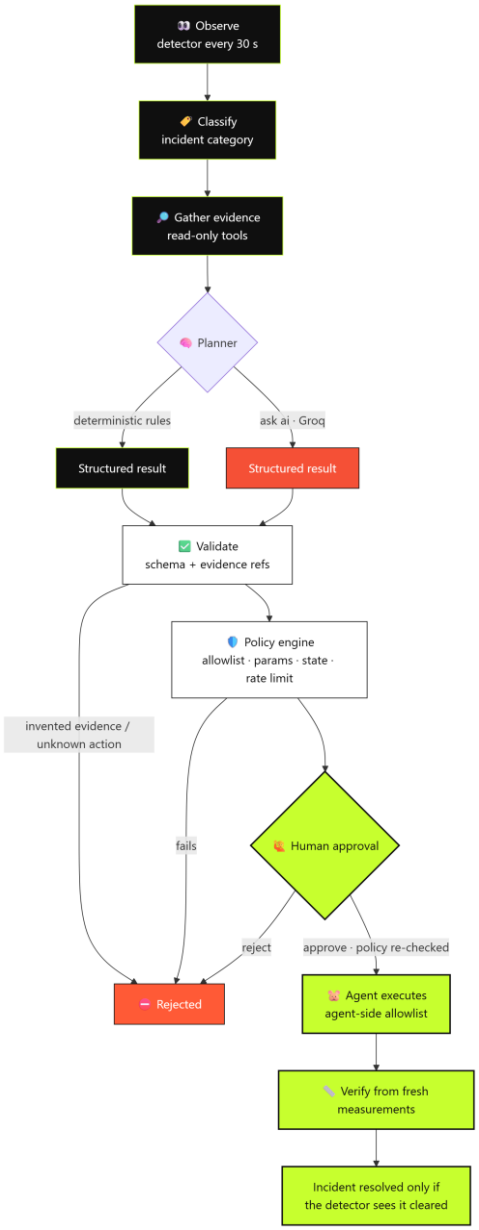


Figure 4 — Observe → evidence → plan → validate → policy → human approval → execute → verify.

4. What is in the project

4.1 Public pages

- **Landing page (/welcome)** — the hero (“your devices. your network. no limits.”), the four pillars, a console illustration with a tunnel readout from a real run, a strip of true product facts (paths, measured RTT, the 180 s health rule, WireGuard’s ChaCha20-Poly1305), and How it works, Features, Security and Get started sections with the real join commands. A single Sign in / Open console button; signed-out visitors to / are sent here.
- **Sign in / sign up (/login)** — brand and tagline panels, the pitch with four features, and a card with sign-in / sign-up tabs, fields with icons, a show-password toggle and a live “systems online” status bar backed by /healthz. The version shown (v0.3.0) is the app’s real version.



Figure 5 — Landing page as a signed-out visitor sees it (headless Chrome, 24 Oct 2026).

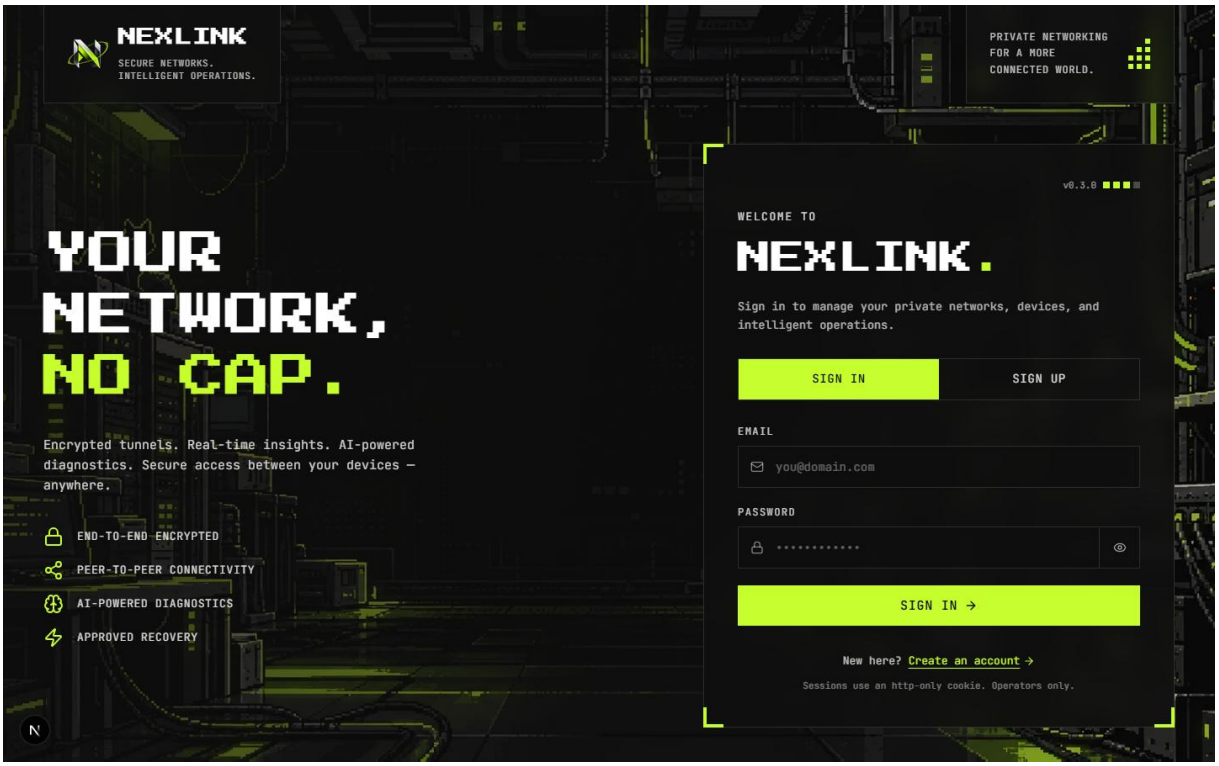


Figure 6 — Sign-in page (headless Chrome, 24 Oct 2026).

4.2 Console

- **Overview** — network health verdict and score computed from records, registered / degraded / unreachable counts, AI briefing per network, approval queue and recent activity.
- **Networks and network detail** — create with CIDR validation and a suggested free range; observed status, address range and the devices in the network.
- **Devices and device page** — identity, connection (agent, interface, peers — separate signals), the network observatory (RTT, loss and throughput over 1 h–7 d with incidents and path changes on the same axis), events, the live agent console, “run on this device” actions and “analyze with AI”.
- **Topology** — React Flow map; solid lime only for links with a fresh handshake, labelled direct or relay; animated packets on verified tunnels.
- **Autopsy** — incidents with investigation states, a path view (control plane → device → peer, with relay when used), diagnostic checks, evidence, AI analysis and proposed recovery.
- **AI Insights** — a technical report per network (summary, numbered findings with evidence references, risks and likely causes with plausible / untested / ruled-out tags, uncertainties) plus analytics: latency, loss, incidents and per-device tunnel quality.
- **Simulation** — an animated, clearly labelled replay of the two-PC protocol with failure scenarios and a packet inspector.
- **Approvals and audit log** — request records with risk and target device; searchable append-only audit trail.
- **Setup and add a device** — guided three-step onboarding and code approval.
- **Navigation** — a sidebar that slides between full width and an icon rail (remembered per browser, Ctrl+B), a slide-in drawer with swipe-to-close on phones, and a “home page” button back to /welcome.

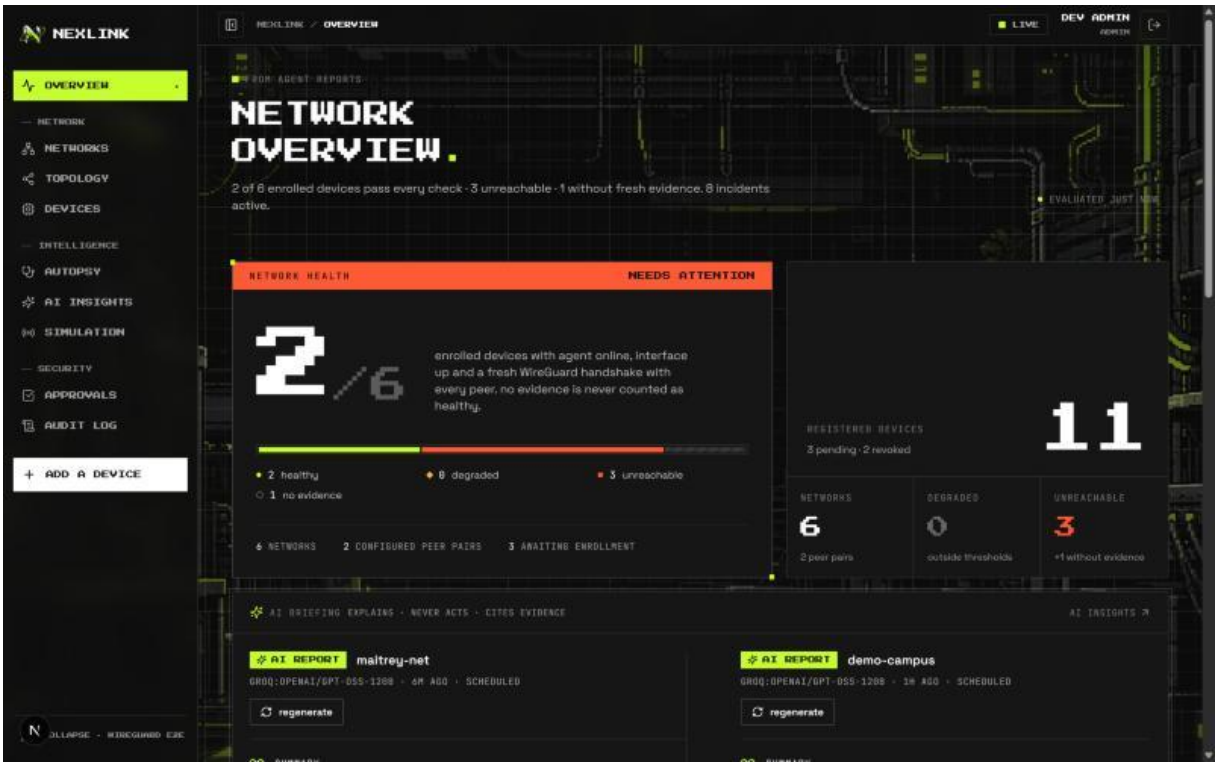


Figure 7 — Console overview with real records from the development database (22 Oct 2026).

4.3 Server (inside the web app)

- Authentication: scrypt password hashing, HttpOnly SameSite=Strict session cookie, CSRF token, lockout, rate limits; sign-up can be disabled with ALLOW_SIGNUP=false.
- Authorization: admin / operator / viewer roles enforced in the backend; out-of-scope resources return 404.
- Networks: private-range CIDR validation, overlap rejection, a suggested free range, address allocation guarded by a unique index.
- Enrollment: device codes with Ed25519 proof of possession and operator approval; invite links whose token rides in the URL fragment; revocation.
- Telemetry, health model, topology, metrics endpoint for the observatory, incident detector, read-only diagnostic tools, deterministic planner, policy engine, approvals, command dispatch, post-execution verification, audit, Socket.IO live updates.
- Relay: one signed WebSocket per agent; same-network forwarding only; per-device bandwidth cap; closed immediately on revocation.
- AI: built-in Groq provider; scheduled and on-request network briefings; incident diagnosis and device analysis; schema and evidence-grounding validation; private keys, passwords and tokens are never sent.

4.4 Networking agent (Go)

Command	What it does
join	Creates the computer’s keys, requests a device code, shows it, waits for approval, then starts the tunnel
run --userspace	Heartbeats, telemetry, probes and approved commands, plus its own WireGuard tunnel (no WireGuard app, no Administrator). `--relay` / `--force-relay` control the fallback; `--share rdp,ssh` offers services for remote sessions
session	`session --list` shows devices, online state and path; `session <device> rdp` opens Remote Desktop through the tunnel

Command	What it does
enroll	Enrolls with a one-time token (read from a prompt, never a flag)
install / uninstall / service-status	Windows service that starts with the PC; output still reaches the console
status / doctor	Local inspection and a read-only preflight
export-config	Tunnel file for the WireGuard app (only /32 peer routes, no DNS, no default route)
lab	Two enrolled agents on one host over loopback, for rehearsals
probe / apply / version	Bounded TCP probe; explicit privileged apply with read-back; version

4.5 Diagnostic service (Python, optional)

The FastAPI service from the first version is still supported (set DIAGNOSTIC_SERVICE_URL), with mock and Groq providers and the same grounding rules. The web app no longer needs it: with GROQ_API_KEY set, AI runs inside the main process.

5. Status matrix

Legend: Done & tested · Partially verified · Known gap · Ready, manual run pending · Not started

Area	Status	Evidence
Login, sessions, CSRF, lockout, RBAC, scoped access	Done & tested	Web test suite + browser check of the new sign-in page
Networks: CIDR, overlap, suggested range, unique addresses	Done & tested	Test suite + console
Device-code enrollment (join with a code)	Done & tested	device-code.test.js + real agent, 22 Oct
Invite links and one-time tokens	Done & tested	Test suite
Signed agent protocol (Go ↔ Node)	Done & tested	Shared signature test vector
Heartbeats, telemetry, honest health model	Done & tested	Test suite; console waited for evidence before showing healthy
Topology and observatory	Done & tested	Browser checks with a live tunnel
Incidents → diagnosis → policy → approval → verify	Done & tested	Test suite; approved test 3/3 valid
Encrypted relay	Done & tested	relay.test.js; real tunnel over the relay, 22 Oct
Direct UDP tunnel between two agent processes	Done & tested	maitrey ⇄ maitrey-peer, 0.35–0.48 ms, 22 and 24 Oct
Built-in AI: briefings, diagnosis, device analysis	Done & tested	ai-provider.test.js, insights.test.js + live Groq runs
Landing page, sign-in page, design system	Done & tested	Production build; fonts and overflow checked per page at desktop and phone sizes
Remote sessions (RDP / SSH through the tunnel)	Ready, manual run pending	Built (agent session command, --share); not yet run end to end in these checks
Upgrade from a working relay link to direct	Known gap	Observed 22 Oct; see section 8
Tunnel between two physical PCs / different networks	Ready, manual run pending	Same agent and join line; needs a second PC and a deployed portal
Kernel / WireGuardNT interface (WireGuard app)	Ready, manual run pending	Needs WireGuard app + Administrator
MFA, SSO, password reset, OS keychain keys, CI	Not started	Roadmap

6. Verification results

6.1 Automated tests

Suite	Command	Result	Notes
Web app	cd web && npm test	85 / 85	Integration tests against an ephemeral MongoDB; real keys for the simulated agents (24 Oct)
Go agent	cd networking-agent && go test ./...	28 / 28	Unit tests and a real wireguard-go tunnel on loopback (24 Oct)
Diagnostic service	python -m pytest -q	15 / 15	Contracts with Groq mocked (24 Oct)
Production build	cd web && npm run build	Passed	19 routes (22 Oct)

6.2 Real WireGuard tunnel with real names

Setup: in the console, network **maitrey-net** (10.42.0.0/24) was created. On this PC (hostname Maitrey) the agent's `join` command was run twice with separate state folders; each printed a short code and the computer name "Maitrey". Both codes were approved on **add a device** and named **maitrey** (10.42.0.2) and **maitrey-peer** (10.42.0.3). Both were then started with the production command `run --userspace` on UDP ports 51840 and 51841 — not the `lab` shortcut.

Check	Observed result
Enrollment	Both agents enrolled after operator approval; addresses allocated by the server
First start (both at once)	No published addresses yet, so each peer started on the relay: handshake 1 s after start; probes through the tunnel 3.06 ms and 3.49 ms, later 0.99 ms
Console during the first minute	"Last handshake: never", devices degraded — correct: the first report was sampled just before the handshake completed
Console after the next report	maitrey-net OPERATIONAL, "1/1 peer pairs verified by fresh handshake"; both devices healthy, latency 1.0 ms
Restart of maitrey-peer	Picked maitrey's LAN address: path direct, handshake 1 s, RTT 0.35 ms (22 Oct)
Re-check on 24 Oct	Both agents restarted: path direct, fresh handshake, RTT 0.42–0.48 ms

Scope of this result

It proves the production path — device-code enrollment → desired config → WireGuard handshake → probes through the tunnel → telemetry → console verdict — over both the relay and the direct path, using the real WireGuard protocol and real names. Both agents ran on one PC, so it does not yet prove connectivity between two physical machines or across different networks; that is the pending two-PC test.

6.3 Browser checks

- Every console route, the landing page and the sign-in page render the three typefaces as intended (pixel titles, Space Grotesk interface text, JetBrains Mono data) with no browser-default fonts.
- No page scrolls sideways at 1100 × 700, 1440 × 900 or 390 × 844.
- Sign-in and sign-up tabs, the show-password toggle and error display (wrong password → INVALID_CREDENTIALS) work on the new sign-in page; the redirect after sign-in is unchanged.

- Sidebar collapse and expand, Ctrl+B, the remembered choice after reload, the mobile drawer (open, swipe to close, Escape) and the “home page” button back to /welcome.
- The landing page shows “Sign in” to signed-out visitors and “Open console” when signed in; its status chip follows /healthz.

6.4 Bugs found and fixed during verification

Bug	Fix
Element resets overrode Tailwind utilities (white text on a white “ask ai” button)	Moved the resets into the CSS base layer so utility classes win
Dev hot-reload socket blocked other WebSockets	server.js routes /_next/ upgrades to Next’s own handler
Topology packet drawn at the origin before its animation began	Packets stay hidden until their start time
Grids overflowed on phones	Explicit minmax(0, 1fr) base columns
“ask ai” button had no accessible name	aria-label added
The whole package.json would have shipped to the browser	Only the version is passed through the Next config
Landing headline wrapped beside the sidebar	Sizes adjusted; later the landing sidebar was removed
Relay health check could report a state it had not observed	check_relay_health now reports the relay’s real state

7. Security model

- Sessions in HttpOnly, SameSite=Strict cookies; CSRF token kept in memory only; scrypt hashing; lockout and rate limits.
- Every agent request — and the relay socket — is Ed25519-signed over method, path, timestamp, nonce, device id and body hash; nonces are single-use.
- Private keys are generated on the device and never stored on the server. Device codes require proof of possession and an operator’s approval; invite tokens are single-use, hashed at rest and travel after the URL “#”.
- The relay forwards only WireGuard ciphertext, only within one network, and is cut the moment a device is revoked.
- RBAC and network scope enforced in the backend; strict schemas reject unknown fields.
- The agent applies only /32 peer routes inside the network range; privileged mode is a local opt-in.
- AI output is untrusted input: schema, evidence grounding, action allowlist, policy engine, human approval, agent-side allowlist. The AI never receives private keys, passwords or session tokens.
- The public pages expose nothing private: the landing page shows no live counts, and only the app version is sent to the browser.
- Audit log is append-only from the API’s point of view; secret-like metadata is redacted.

8. What is not in the project yet

Item	Status	What is needed / next step
Tunnel between two physical PCs / across networks	Ready, manual run pending	Deploy the portal (docs/deploy.md) or use a second PC on the same Wi-Fi; run the join line there
Relay → direct upgrade	Known gap	Periodically retry direct while on the relay, and switch only after a new handshake proves the direct path (networking-agent/cmd/nexlink-agent/userspace.go)
Kernel / WireGuardNT interface	Ready, manual run pending	Install WireGuard for Windows; import export-config output; run as Administrator
Application-level checks	Not started	Service probes defined per network
MFA, SSO, password reset	Not started	Account hardening; the sign-in page has no buttons for these until they exist
Agent keys in OS keychain / TPM	Not started	Today keys are files protected by file permissions
CI pipeline	Not started	Run the three test suites on every push
Multi-instance scaling	Not started	Shared rate-limit store, Socket.IO adapter, relay across instances

Known limitations and environment notes

- If two devices start on the relay at the same moment, they stay on the relay while it works. Traffic is still end-to-end encrypted, but it passes through the server and is slightly slower. Restarting one agent once the other has published its address gives a direct path.
- The userspace tunnel carries NexLink traffic (handshakes, probes, approved tests and sessions), not every application on the PC; a system-wide adapter uses the WireGuard app path.
- Plain HTTP between agent and server is allowed only on loopback or a lab LAN; deploy behind HTTPS for anything shared.
- Docker Desktop on the development PC fails to start (its inference socket path cannot be accessed). The project does not depend on Docker for development.
- The development database still contains demo and test networks; use a fresh database for presentations.

9. How to run

Development (one window)

```
cd web
npm install
npm run dev        # http://localhost:3000 — embedded MongoDB if MONGODB_URI is not set
```

Open http://localhost:3000. Signed-out visitors see the landing page; **Get started** opens sign-up, and the first account becomes admin. Set GROQ_API_KEY in web\.env to enable AI.

Add a computer

```
irm 'http://<portal address>:3000/join.ps1' | iex        # Windows
curl -fsSL 'http://<portal address>:3000/join.sh' | sh    # macOS / Linux
```

It shows a short code; approve it on **add a device**. On the PC running the portal the agent can also be started directly:

```
cd networking-agent\bin
.\nexlink-agent.exe join --server http://127.0.0.1:3000 --state-dir .\pc-1
.\nexlink-agent.exe run --userspace --server http://127.0.0.1:3000 --state-dir .\pc-1
```

Deploy

One container (root Dockerfile) and one MongoDB: Docker Compose behind Caddy for HTTPS, or a PaaS such as Render, Railway or Fly.io with MONGODB_URI, PUBLIC_URL and TRUST_PROXY=true. Full steps in docs/deploy.md.

Tests

```
cd web && npm test
cd networking-agent && go test ./...
cd diagnostic-service && python -m pytest -q
```

Roadmap

1. Run the two-PC test, then a cross-network test through a deployed portal.
2. Upgrade working relay links to direct once a direct path is proven.
3. Add CI, MFA and OS-keychain key storage.
4. Application-level checks per network.

Appendix A — Repository layout

Path	Contents
web/	The whole platform: Next.js pages (app/), components, features, and the server (server/: modules, database models, relay, AI) started by server.js; tests/
networking-agent/	Go agent: cmd/nexlink-agent and internal/* packages; prebuilt binaries in bin/
diagnostic-service/	Optional FastAPI AI service: schemas, providers (mock, Groq), validators; tests/
scripts/	start-all, build-agent (all OS downloads), demo-data, lan-firewall
docs/	Architecture, protocols, API contracts, database schema, threat model, deploy and run guides, this report and the demo guide
Dockerfile, docker-compose.yml	One container for the web app; compose adds MongoDB

Appendix B — Documentation index

Document	What is inside
docs/run-commands.md	Starting NexLink by hand, adding computers, remote sessions, troubleshooting
docs/deploy.md	Putting NexLink on the internet: Compose + Caddy, PaaS, settings
docs/architecture.md	Components, health model, diagnostic loop, MongoDB constraints
docs/network-protocols.md	Enrollment, request signing, telemetry, probes, relay
docs/api-contracts.md	Every REST endpoint with its permission
docs/database-schema.md	Collections, indexes, retention
docs/threat-model.md	Threats, mitigations, known gaps
docs/two-pc-test.md	Runbook for the two-PC tunnel
docs/NexLink-Demo-Guide	Step-by-step script for presenting the project